

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Database Management Systems (BCSC-1003)

TOPIC: RELATIONAL ALGEBRA IN DBMS

INTRODUCTION

- THERE ARE TWO TYPES OF QUERY LANGUAGES:
  - PROCEDURAL QL
  - NON-PROCEDURAL QL
- PROCEDURAL
  - WHAT TO RETRIEVE AND HOW TO RETRIEVE.
  - E.G. RELATIONAL ALGEBRA
  - RESULTS ALWAYS DISTINCT TUPLES
  - OPERATIONS: Σ, $\prod$, $\cup$, AND SO ON
- NON-PROCEDURAL
  - WHAT TO RETRIEVE.
  - E.G. TUPLE RELATIONAL CALCULUS, SQL
  - TUPLE RELATIONAL CALCULUS: SYMBOLS USED IN FIRST ORDER LOGIC.
  - SQL: SELECT * FROM TABLE;

RELATIONAL ALGEBRA

- RELATIONAL ALGEBRA IS A WIDELY USED PROCEDURAL QUERY LANGUAGE.
- IT TAKES INSTANCES OF RELATIONS AS INPUT AND YIELDS INSTANCES OF RELATIONS AS OUTPUT.
- IT USES VARIOUS OPERATIONS TO PERFORM THIS ACTION.
- RELATIONAL ALGEBRA MAINLY PROVIDES THEORETICAL FOUNDATION FOR RELATIONAL DATABASES AND SQL.

BASIC SQL RELATIONAL ALGEBRA OPERATIONS

RELATIONAL ALGEBRA MAY BE DIVIDED IN VARIOUS GROUPS:

1. RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY
   - A. UNION (∪)
   - B. INTERSECTION (∩)
   - C. DIFFERENCE (–)

BASIC SQL RELATIONAL ALGEBRA OPERATIONS

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

2. UNARY RELATIONAL OPERATIONS

A. SELECT (Σ)

B. PROJECT (Π)

C. RENAME (P)

BASIC SQL RELATIONAL ALGEBRA OPERATIONS

3. BINARY RELATIONAL OPERATIONS

I. **JOIN (⋈)**

TYPES OF JOIN:

VARIOUS FORMS OF JOIN OPERATION ARE:

A. CROSS JOIN OR CARTESIAN PRODUCT

B. INNER JOINS:

(I). THETA JOIN **(II). EQUI JOIN** **(III). NATURAL JOIN**

C. OUTER JOIN:

(I). LEFT OUTER JOIN **(II). RIGHT OUTER JOIN** **(III). FULL OUTER JOIN**

II. DIVISION (÷)

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY

UNION (∪)

UNION IS SYMBOLIZED BY ∪ SYMBOL. IT INCLUDES ALL TUPLES THAT ARE IN TABLES A OR IN B. IT ALSO ELIMINATES DUPLICATE TUPLES. SO, SET A UNION SET B WOULD BE EXPRESSED AS:

$R = A \cup B$

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY

UNION (∪)

EXP.

Relation A

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT

Relation B

CNAME	CSTATUS
KARAN	POOR
RAJAT	GOOD

R = A ∪ B

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT
KARAN	POOR

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY

INTERSECTION (∩)

INTERSECTION IS SYMBOLIZED BY ∩ SYMBOL. DEFINES A RELATION CONSISTING OF A SET OF ALL TUPLE THAT ARE IN BOTH A AND B. HOWEVER, A AND B MUST BE UNION-COMPATIBLE.

$R = A \cap B$

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY

INTERSECTION (∩)

EXP.

Relation A

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT

Relation B

CNAME	CSTATUS
KARAN	POOR
RAJAT	GOOD

$R = A \cap B$

CNAME	CSTATUS
RAJAT	GOOD

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY

DIFFERENCE (–)

DIFFERENCE IS SYMBOLIZED BY – SYMBOL. THE RESULT OF A–B, IS A RELATION WHICH INCLUDES ALL TUPLES THAT ARE IN A BUT NOT IN B.

R = A – B

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY

DIFFERENCE (–)

EXP.

Relation A

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT

Relation B

CNAME	CSTATUS
KARAN	POOR
RAJAT	GOOD

R = A – B

CNAME	CSTATUS
RAHUL	EXCELLENT

UNARY RELATIONAL OPERATIONS

A. SELECT (Σ)

B. PROJECT (Π)

C. RENAME (P)

SELECT OPERATOR (Σ)

- SELECT operation is used for selecting a subset of the tuples according to a given selection condition.
- Select Operator is denoted by sigma (σ).
- It is used as an expression to choose tuples which meet the selection condition.
- Select operator selects tuples that satisfy a given predicate.

SELECT OPERATOR (Σ)

Syntax:

$\sigma_{<Condition>}$ (Relation/Table name)

Properties:

1. It is like a WHERE clause in SQL.
2. Selection operator is commutative i.e.,

$$\sigma_{<condition_1>}(\sigma_{<condition_2>}(R)) = \sigma_{<condition_2>}(\sigma_{<condition_1>}(R))$$

SELECT OPERATOR (Σ)

Examples: STUDENT

ROLL_NO	NAME	AGE	COURSE
1	ABHISHEK	17	BTech
2	AMIT	16	BCA
3	AJEET	17	BTech
4	AKHIL	18	BTech
5	PRASHANT	17	BCA

Query 1: Find out the stude

Syntax: $\sigma_{\text{course} = \text{'BTech'}}$ (student)

Query2: Find the student(s) whose age is greater than 17.

Syntax: $\sigma_{\text{age} > 17}$ (student)

PROJECT OR PROJECTION OPERATOR (Π)

- PROJECT Operator is denoted by pi (π).
- Project operation selects certain attributes discarding other attributes.
- It is also known as vertical partitioning since it partitions the relation or table vertically.
- Duplicate rows are automatically eliminated, as relation is a set.

PROJECT OPERATOR (Π)

Syntax:

$\pi_{<\text{Attribute list}>}$ (Relation/Table name)

Properties:

1. The degree of output relation (number of columns present) is equal to the number of attributes mentioned in the attribute list.
2. Projection operator does not obey commutative property i.e.

$\pi_{<\text{list2}>}(\pi_{<\text{list1}>}(R)) \neq \pi_{<\text{list1}>}(\pi_{<\text{list2}>}(R))$

PROJECT OPERATOR (Π)

Examples: STUDENT

ROLL_NO	NAME	AGE	COURSE
1	ABHISHEK	17	BTech
2	AMIT	16	BCA
3	AJEET	17	BTech
4	AKHIL	18	BTech
5	PRASHANT	17	BCA

Query 1: Find out the name and course from STUDENT relation.

Syntax: $\pi_{\text{name, course}}$ (student)

Query 2: Find the age from STUDENT relation.

Syntax: π_{age} (student)

AGE
17
16
18

RENAME OPERATOR (P)

- The results of relational algebra are also relations but without any name.
- The RENAME operation allows us to rename the relation.
- It is denoted by rho (ρ).

RENAME Operator (ρ)

Syntax:

ρ (Relation2, Relation1)

Examples:

Query 1: Rename STUDENT relation to STUDENT1 relation.

Syntax: ρ (STUDENT1, STUDENT)

Query 2: Create a relation STUDENT_NAMES with RNO and NAME from STUDENT.

Syntax: ρ (STUDENT_NAMES, $\pi_{(RNO, NAME)}$ (STUDENT))

BINARY RELATIONAL ALGEBRA OPERATIONS

3. BINARY RELATIONAL OPERATIONS

I. JOIN (⋈)

TYPES OF JOIN:

VARIOUS FORMS OF JOIN OPERATION ARE:

A. CROSS JOIN OR CARTESIAN PRODUCT

B. INNER JOINS:

(I). THETA JOIN (II). EQUI JOIN (III). NATURAL JOIN

C. OUTER JOIN:

(I). LEFT OUTER JOIN (II). RIGHT OUTER JOIN

(III). FULL OUTER JOIN

II. DIVISION (÷)

JOIN (⋈)

- JOIN IS A BINARY OPERATION WHICH ALLOWS US TO COMBINE JOIN PRODUCT AND SELECTION IN ONE SINGLE STATEMENT.

- THE GOAL OF CREATING A JOIN CONDITION IS THAT IT HELPS US TO COMBINE THE DATA FROM TWO OR MORE DBMS TABLES.

- VARIOUS FORMS OF JOIN OPERATION ARE: CROSS JOIN, INNER JOIN & OUTER JOIN.

CROSS JOIN OR CARTESIAN PRODUCT (×)

CROSS JOIN OR CARTESIAN PRODUCT IS SYMBOLIZED BY × SYMBOL. IT IS AN OPERATION USED TO MERGE COLUMNS FROM TWO RELATIONS. IT IS ALSO CALLED CROSS PRODUCT.

$R = A \times B$

CROSS JOIN OR CARTESIAN PRODUCT (×)

EXP.

Relation A

NAME	AGE
KAJAL	32
ANIL	40

Relation B

JOB	LOCATION
DEVELOPER	CHENNAI
ANALYST	MUMBAI

$R = A \times B$

NAME	AGE	JOB	LOCATION
KAJAL	32	DEVELOPER	CHENNAI
KAJAL	32	ANALYST	MUMBAI
ANIL	40	DEVELOPER	CHENNAI
ANIL	40	ANALYST	MUMBAI

INNER JOIN

- INNER JOIN IS USED TO RETURN ROWS FROM BOTH TABLES WHICH SATISFY THE GIVEN CONDITION.
- IT IS THE MOST WIDELY USED JOIN OPERATION AND CAN BE CONSIDERED AS A DEFAULT JOIN-TYPE.
- INNER JOIN FURTHER DIVIDED INTO THREE SUBTYPES:

(I). THETA JOIN (II). EQUI JOIN (III). NATURAL JOIN

INNER JOIN: THETA JOIN

- THETA JOIN ALLOWS US TO MERGE TWO TABLES BASED ON THE CONDITION REPRESENTED BY THETA.
- THETA JOINS WORK FOR ALL COMPARISON OPERATORS.
- IT IS DENOTED BY SYMBOL Θ.
- THE GENERAL CASE OF JOIN OPERATION IS CALLED A THETA JOIN.

SYNTAX: $\mathbf{A} \bowtie_{\Theta} \mathbf{B}$

R1 AND R2 ARE RELATIONS HAVING ATTRIBUTES (A_1, A_2, .., A_N) AND (B_1, B_2, ..., B_N) SUCH THAT THE ATTRIBUTES DON'T HAVE ANYTHING IN COMMON, THAT IS R1 ∩ R2 = Φ.

INNER JOIN: THETA JOIN

EXAMPLE:

STUDENT

SID	NAME	STD
1001	AJAY	11
1002	JATIN	12

SUBJECT

CLASS	SUBJECT
11	MATH
11	HINDI
12	COMPUTER
12	SCIENCE

$$\text{STUDENT} \bowtie_{\text{STUDENT.STD} = \text{SUBJECT.CLASS}} \text{SUBJECT}$$

OUTPUT:

SID	NAME	STD	CLASS	SUBJECT
1001	AJAY	11	11	MATH
1001	AJAY	11	11	HINDI
1002	JATIN	12	12	COMPUTER
1002	JATIN	12	12	SCIENCE

INNER JOIN: EQUI JOIN

- EQUI JOIN IS DONE WHEN A THETA JOIN USES ONLY THE EQUIVALENCE CONDITION.

- WHEN THETA JOIN USES ONLY EQUALITY COMPARISON OPERATOR, IT IS SAID TO BE EQUI JOIN.

- THE PREVIOUS EXAMPLE CORRESPONDS TO EQUI JOIN.

INNER JOIN: NATURAL JOIN

- NATURAL JOIN DOES NOT UTILIZE ANY OF THE COMPARISON OPERATORS.
- IN THIS TYPE OF JOIN, THE ATTRIBUTES SHOULD HAVE THE SAME NAME AND DOMAIN.
- IN NATURAL JOIN, THERE SHOULD BE AT LEAST ONE COMMON ATTRIBUTE BETWEEN TWO RELATIONS.
- IT PERFORMS SELECTION FORMING EQUALITY ON THOSE ATTRIBUTES WHICH APPEAR IN BOTH RELATIONS AND ELIMINATES THE DUPLICATE ATTRIBUTES.

INNER JOIN: NATURAL JOIN

EXAMPLE: R1

NUM	SQUARE
2	4
3	9

R2

NUM	CUBE
2	8
3	27

R1 ⋈ R2

OUTPUT:

NUM	SQUARE	CUBE
2	4	8
3	9	27

OUTER JOIN

- AN OUTER JOIN DOESN'T REQUIRE EACH RECORD IN THE TWO JOIN TABLES TO HAVE A MATCHING RECORD.
- IN THIS TYPE OF JOIN, THE TABLE RETAINS EACH RECORD EVEN IF NO OTHER MATCHING RECORD EXISTS.
- OUTER JOIN FURTHER DIVIDED INTO THREE SUBTYPES:

  (I). LEFT OUTER JOIN

  (II). RIGHT OUTER JOIN

  (III). FULL OUTER JOIN

OUTER JOIN: LEFT OUTER JOIN (⟕)

- LEFT OUTER JOIN RETURNS ALL THE ROWS FROM THE TABLE ON THE LEFT EVEN IF NO MATCHING ROWS HAVE BEEN FOUND IN THE TABLE ON THE RIGHT.
- WHEN NO MATCHING RECORD FOUND IN THE TABLE ON THE RIGHT, NULL IS RETURNED.
- IT IS DENOTED BY SYMBOL (⟕).

SYNTAX: **A ⟕ B**



OUTER JOIN: LEFT OUTER JOIN (⟕)

EXAMPLE:

R1

NUM	SQUARE
2	4
3	9
4	16

R2

NUM	CUBE
2	8
3	27
5	125

R1 ⟕ R2

OUTPUT:

NUM	SQUARE	CUBE
2	4	8
3	9	27
4	16	NULL

OUTER JOIN: RIGHT OUTER JOIN (⟖)

- RIGHT OUTER JOIN RETURNS ALL THE COLUMNS FROM THE TABLE ON THE RIGHT EVEN IF NO MATCHING ROWS HAVE BEEN FOUND IN THE TABLE ON THE LEFT.
- WHERE NO MATCHES HAVE BEEN FOUND IN THE TABLE ON THE LEFT, NULL IS RETURNED. RIGHT OUTER JOIN IS THE OPPOSITE OF LEFT OUTER JOIN.
- IT IS DENOTED BY SYMBOL (⟖).

SYNTAX: **A ⟖ B**



OUTER JOIN: RIGHT OUTER JOIN (⟖)

EXAMPLE:

R1

NUM	SQUARE
2	4
3	9
4	16

R2

NUM	CUBE
2	8
3	27
5	125

R1 ⟖ R2

OUTPUT:

NUM	CUBE	SQUARE
2	8	4
3	27	9
5	125	NULL

OUTER JOIN: FULL OUTER JOIN (⟗)

- IN A FULL OUTER JOIN , ALL TUPLES FROM BOTH RELATIONS ARE INCLUDED IN THE RESULT, IRRESPECTIVE OF THE MATCHING CONDITION.
- IT IS DENOTED BY SYMBOL (⟗).

SYNTAX: **A ⟗ B**

OUTER JOIN: FULL OUTER JOIN (⟗)

EXAMPLE:

R1

NUM	SQUARE
2	4
3	9
4	16

R2

NUM	CUBE
2	8
3	27
5	125

R1 ⟗ R2

OUTPUT:

NUM	SQUARE	CUBE
2	4	8
3	9	27
4	16	NULL
5	NULL	125

DIVISION (÷)

DIVISION OPERATOR A÷B CAN BE APPLIED IF AND ONLY IF:

- ATTRIBUTES OF B IS PROPER SUBSET OF ATTRIBUTES OF A.
- THE RELATION RETURNED BY DIVISION OPERATOR WILL HAVE ATTRIBUTES = (ALL ATTRIBUTES OF A – ALL ATTRIBUTES OF B).
- THE RELATION RETURNED BY DIVISION OPERATOR WILL RETURN THOSE TUPLES FROM RELATION A WHICH ARE ASSOCIATED TO EVERY B'S TUPLE.

SYNTAX: **A ÷ B**

DIVISION (÷)

EXAMPLE:

RELATION A HAS 1 TABLE AS:

Sno	Pno
S1	P1
S1	P2
S1	P3
S1	P4
S1	P5
S1	P6
S2	P1
S2	P2
S3	P2
S4	P2
S4	P4
S4	P5

Relation B has 3 tables as:

Pno
P1

Pno
P2
P4

Pno
P1
P2
P3
P4
P5
P6

Then, A DIVIDE BY B gives the following resultant tables for all the three cases as follows:

Sno
S1
S2

Sno
S1
S4

Sno
S1

EXERCISE

Example 1: Consider the following schema:
SUPPLIER (Sid, S_name, S_addr)
PARTS (Pid, P_name, color)
CATALOG (sid, pid, cost)

Now, answer the following queries in Relational Algebra:
(a) Find the name of all the suppliers who supply Yellow parts.
(b) Find the Sid of suppliers who supply every parts.
(c) Find the Sid of suppliers who supply every red or green part.

EXERCISE

Solution:

(a) Find the name of all the suppliers who supply Yellow parts.

$$\pi_{S_name}(SUPPLIER) \bowtie CATALOG \bowtie (\sigma_{color = \text{'Yellow'}}(PARTS))$$

(b) Find the Sid of suppliers who supply every parts.

$$\pi_{sid,\ pid}(CATALOG) \div \pi_{pid}(PARTS)$$

(c) Find the Sid of suppliers who supply every red or green part.

$$(\pi_{sid,\ pid}(CATALOG)) \div (\pi_{pid}(\sigma_{color = \text{'red'} \vee color = \text{'green'}}(PARTS)))$$

EXERCISE

Example 2: Consider a database that has the relation schema CR (StudentName, CourseName). An instance of the schema CR is as given below.

StudentName	CourseName
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

The following queries are made on the database.

$T1 \leftarrow \pi_{CourseName}(\sigma_{StudentName='SA'}(CR))$

$T2 \leftarrow CR \div T1$

The number of rows in T2 are _____.

(GATE 2017)

EXERCISE

Solution:

T1 will give:

CourseName
CA
CB
CC

T2←CR÷T1 = All the tuples in CR which are matched with every tuple in T1:

StudentName
SA
SC
SD
SF

REFERENCE BOOKS

1. Elmasri and Navathe (2010), “Fudamentals of Database Systems”, 5th Edition, Addision Wesley.
2. Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
3. Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
4. M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.

THANK YOU !!!